Android Studio Mobile Application Development

Comprehensive Java + XML Code Snippets Notes

Focus: Practical Android development using Java and XML layouts

Topics covered

- Form widgets and text fields
- Text-field validation
- Layouts and common layout patterns
- Adapters, ArrayAdapter, ListView and Spinner
- AlertDialog, Toast and PopupWindow
- Fragments
- Time and Date
- SharedPreferences
- Explicit and Implicit Intents

Java + XML • Lab-oriented • Reusable snippets • KTU/MCA reference

Contents

1. Form widgets and text fields
2. Text-field validation
3. Layouts and common layout patterns
4. Adapters, ArrayAdapter, ListView and Spinner
5. AlertDialog, Toast and PopupWindow
6. Fragments
7. Time and Date
8. SharedPreferences
9. Explicit and Implicit Intents

Quick reference conventions

Examples use a simple project structure with an Activity written in Java and XML layouts in res/layout. Replace package names, IDs and resource names to match your project.

> Important: These notes focus on classic Views/XML because that is the requested Java + XML approach. AndroidX classes are used where appropriate.

1. Form Widgets and Text Fields

Form widgets are UI controls used to accept input or let the user make selections. The most common XML widgets are TextView, EditText, Button, CheckBox, RadioButton/RadioGroup, Switch and ImageButton.

Basic form XML — activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Registration Form"
        android:textSize="24sp" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter name"
        android:inputType="textPersonName" />

    <EditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter email"
        android:inputType="textEmailAddress" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword" />

    <CheckBox
        android:id="@+id/cbTerms"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="I accept the terms" />

    <RadioGroup
        android:id="@+id/rgGender"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <RadioButton
            android:id="@+id/rbMale"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Male" />

        <RadioButton
            android:id="@+id/rbFemale"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Female" />
    </RadioGroup>

    <Switch
        android:id="@+id/swNotifications"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Notifications" />

    <Button
        android:id="@+id/btnSubmit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Submit" />
</LinearLayout>
```

Accessing widgets and reading values — MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    EditText etName, etEmail, etPassword;
    CheckBox cbTerms;
```

```
    RadioGroup rgGender;
    Switch swNotifications;
    Button btnSubmit;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etName = findViewById(R.id.etName);
        etEmail = findViewById(R.id.etEmail);
        etPassword = findViewById(R.id.etPassword);
        cbTerms = findViewById(R.id.cbTerms);
        rgGender = findViewById(R.id.rgGender);
        swNotifications = findViewById(R.id.swNotifications);
        btnSubmit = findViewById(R.id.btnSubmit);

        btnSubmit.setOnClickListener(v -> {
            String name = etName.getText().toString().trim();
            String email = etEmail.getText().toString().trim();
            String password = etPassword.getText().toString();

            boolean termsAccepted = cbTerms.isChecked();
            int selectedId = rgGender.getCheckedRadioButtonId();
            String gender = "";

            if (selectedId != -1) {
                RadioButton rb = findViewById(selectedId);
                gender = rb.getText().toString();
            }

            boolean notifications = swNotifications.isChecked();
        });
    }
}
```

Useful widget methods

EditText: getText(), setText(), setError(), requestFocus(). CheckBox/Switch: isChecked(), setChecked(). RadioGroup: getCheckedRadioButtonId(). Button: setOnClickListener().

Input types

Common inputType values

```
android:inputType="text"
android:inputType="textPersonName"
android:inputType="textEmailAddress"
android:inputType="textPassword"
android:inputType="number"
android:inputType="numberDecimal"
android:inputType="phone"
android:inputType="date"
android:inputType="time"
```

2. Validation of Text Fields

Validate input before processing or saving it. A good pattern is: read → trim → check empty → check format/range → show error → stop if invalid.

Required fields + password length

```
private boolean validateBasic() {
    String name = etName.getText().toString().trim();
    String email = etEmail.getText().toString().trim();
    String password = etPassword.getText().toString();

    etName.setError(null);
    etEmail.setError(null);
    etPassword.setError(null);

    if (name.isEmpty()) {
        etName.setError("Name is required");
        etName.requestFocus();
        return false;
    }

    if (email.isEmpty()) {
        etEmail.setError("Email is required");
        etEmail.requestFocus();
        return false;
    }
```

```
    if (password.length() < 6) {
        etPassword.setError("Minimum 6 characters");
        etPassword.requestFocus();
        return false;
    }

    return true;
}
```

Email validation

```
private boolean isValidEmail(String email) {
    return Patterns.EMAIL_ADDRESS.matcher(email).matches();
}

// Usage:
if (!isValidEmail(email)) {
    etEmail.setError("Enter a valid email address");
    etEmail.requestFocus();
    return;
}
```

Number/range validation

```
String ageText = etAge.getText().toString().trim();

if (ageText.isEmpty()) {
    etAge.setError("Age is required");
    return;
}

try {
    int age = Integer.parseInt(ageText);

    if (age < 18 || age > 60) {
        etAge.setError("Age must be between 18 and 60");
        return;
    }
} catch (NumberFormatException e) {
    etAge.setError("Enter a valid number");
}
```

Confirm password

```
String p1 = etPassword.getText().toString();
String p2 = etConfirmPassword.getText().toString();

if (!p1.equals(p2)) {
    etConfirmPassword.setError("Passwords do not match");
    etConfirmPassword.requestFocus();
    return;
}
```

3. Layouts

Layouts control the position and size of views. For XML-based beginner applications, LinearLayout, ConstraintLayout, FrameLayout, ScrollView and TableLayout are especially useful.

LinearLayout — vertical form

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Name" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Save" />
</LinearLayout>
```

LinearLayout — horizontal row with weights

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```
    android:orientation="horizontal">

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="YES" />

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="NO" />
</LinearLayout>
```

ScrollView — long form

```
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- Put form views here -->

    </LinearLayout>
</ScrollView>
```

TableLayout — 4 × 4 style grid

```
<TableLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:stretchColumns="*">

    <TableRow>
        <Button android:text="1" />
        <Button android:text="2" />
        <Button android:text="3" />
        <Button android:text="4" />
    </TableRow>

    <TableRow>
        <Button android:text="5" />
        <Button android:text="6" />
        <Button android:text="7" />
        <Button android:text="8" />
    </TableRow>

    <!-- Add two more TableRows for 4 × 4 -->
</TableLayout>
```

Layout rules to remember

• match_parent = occupy available parent space; wrap_content = size based on content.

• dp is used for layout dimensions; sp is used for text size.

• Avoid deeply nested layouts when possible.

• A ScrollView normally contains one direct child; put a LinearLayout inside it for multiple form controls.

• ConstraintLayout is useful for flexible, responsive positioning; use constraints instead of relying on absolute positions.

4. Android Adapters

An Adapter connects a data source to an AdapterView such as ListView or Spinner. ArrayAdapter is enough for simple strings; custom adapters are used when each row has multiple views.

ArrayAdapter + ListView — XML

```
<ListView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

ArrayAdapter + ListView — Java

```
ListView listView = findViewById(R.id.listView);

String[] names = {"Arun", "Binu", "Cathy", "Deepa"};

ArrayAdapter<String> adapter =
        new ArrayAdapter<>(
            this,
            android.R.layout.simple_list_item_1,
            names
        );

listView.setAdapter(adapter);

listView.setOnItemClickListener((parent, view, position, id) -> {
    String selected = (String) parent.getItemAtPosition(position);
    Toast.makeText(this, selected, Toast.LENGTH_SHORT).show();
});
```

Spinner — XML

```
<Spinner
    android:id="@+id/spinnerCourse"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Spinner — Java

```
Spinner spinner = findViewById(R.id.spinnerCourse);

String[] courses = {"MCA", "MSc CS", "BCA", "BTech"};

ArrayAdapter<String> adapter = new ArrayAdapter<>(
        this,
        android.R.layout.simple_spinner_item,
        courses
);

adapter.setDropDownViewResource(
        android.R.layout.simple_spinner_dropdown_item
);

spinner.setAdapter(adapter);

spinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view,
                               int position, long id) {
        String course = parent.getItemAtPosition(position).toString();
    }

    @Override
    public void onNothingSelected(AdapterView<?> parent) {
    }
});
```

Custom ListView row — XML

```
<!-- row_student.xml -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="12dp">

    <TextView
        android:id="@+id/tvName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvCourse"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</LinearLayout>
```

Custom ArrayAdapter — Java

```
public class StudentAdapter extends ArrayAdapter<Student> {

    public StudentAdapter(Context context, List<Student> students) {
        super(context, 0, students);
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        if (convertView == null) {
            convertView = LayoutInflater.from(getContext())
```

```
                    .inflate(R.layout.row_student, parent, false);
        }

        Student s = getItem(position);

        TextView tvName = convertView.findViewById(R.id.tvName);
        TextView tvCourse = convertView.findViewById(R.id.tvCourse);

        tvName.setText(s.getName());
        tvCourse.setText(s.getCourse());

        return convertView;
    }
}
```

5. AlertDialog, Toast and PopupWindow

Basic Toast

```
Toast.makeText(
    MainActivity.this,
    "Saved successfully",
    Toast.LENGTH_SHORT
).show();
```

AlertDialog — message

```
new AlertDialog.Builder(this)
    .setTitle("Delete")
    .setMessage("Do you really want to delete this item?")
    .setPositiveButton("Yes", (dialog, which) -> {
        // delete operation
    })
    .setNegativeButton("No", null)
    .show();
```

AlertDialog — input field

```
EditText input = new EditText(this);
input.setHint("Enter your name");

new AlertDialog.Builder(this)
    .setTitle("Enter Name")
    .setView(input)
    .setPositiveButton("OK", (dialog, which) -> {
        String name = input.getText().toString().trim();
        Toast.makeText(this, name, Toast.LENGTH_SHORT).show();
    })
    .setNegativeButton("Cancel", null)
    .show();
```

AlertDialog — single-choice list

```
String[] colors = {"Red", "Green", "Blue"};

new AlertDialog.Builder(this)
    .setTitle("Choose Color")
    .setSingleChoiceItems(colors, -1, (dialog, which) -> {
        String selected = colors[which];
        Toast.makeText(this, selected, Toast.LENGTH_SHORT).show();
        dialog.dismiss();
    })
    .show();
```

PopupWindow — simple custom popup

```
View popupView = getLayoutInflater()
        .inflate(R.layout.popup_menu, null);

PopupWindow popup = new PopupWindow(
        popupView,
        300,
        ViewGroup.LayoutParams.WRAP_CONTENT,
        true
);

popup.setBackgroundDrawable(
        new ColorDrawable(Color.WHITE)
);
popup.setOutsideTouchable(true);

popup.showAsDropDown(btnSubmit);
```

popup_menu.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="300dp"
    android:layout_height="wrap_content"
    android:orientation="vertical"
```

```
    android:padding="12dp">

    <TextView
        android:id="@+id/tvEdit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Edit"
        android:padding="12dp" />

    <TextView
        android:id="@+id/tvDelete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Delete"
        android:padding="12dp" />
</LinearLayout>
```

6. Fragments

A Fragment is a reusable portion of UI and behavior hosted by an Activity. In XML/Java applications, AndroidX Fragment is the standard choice. A Fragment has its own lifecycle and can have its own XML layout.

fragment_home.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:id="@+id/tvFragment"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Home Fragment"
        android:textSize="24sp" />

    <Button
        android:id="@+id/btnFragment"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click" />
</LinearLayout>
```

HomeFragment.java

```
public class HomeFragment extends Fragment {

    public HomeFragment() {
        super(R.layout.fragment_home);
    }

    @Override
    public void onViewCreated(View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        Button button = view.findViewById(R.id.btnFragment);

        button.setOnClickListener(v ->
            Toast.makeText(requireContext(),
                    "Hello from Fragment",
                    Toast.LENGTH_SHORT).show()
        );
    }
}
```

Activity XML — FragmentContainerView

```
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Add / replace a Fragment from Activity

```
getSupportFragmentManager()
    .beginTransaction()
    .replace(R.id.fragmentContainer, new HomeFragment())
    .addToBackStack(null)
    .commit();
```

Pass data from Activity to Fragment

```
Bundle bundle = new Bundle();
bundle.putString("username", "Deepak");
```

```
HomeFragment fragment = new HomeFragment();
fragment.setArguments(bundle);

getSupportFragmentManager()
    .beginTransaction()
    .replace(R.id.fragmentContainer, fragment)
    .commit();

// Inside Fragment:
String username = requireArguments().getString("username");
```

Important Fragment lifecycle idea

A simplified lifecycle is: onAttach → onCreate → onCreateView/onViewCreated → onStart → onResume → onPause → onStop → onDestroyView → onDestroy → onDetach. UI references should generally be handled with the Fragment view lifecycle.

7. Time and Date

Android provides DatePickerDialog and TimePickerDialog for user selection. For modern Java code, java.time is preferred when the project supports the required API level; otherwise use the Android-compatible APIs available in your project.

DatePickerDialog

```
Calendar calendar = Calendar.getInstance();

DatePickerDialog dateDialog = new DatePickerDialog(
    this,
    (view, year, month, dayOfMonth) -> {
        // month is zero-based: January = 0
        String date = dayOfMonth + "/" + (month + 1) + "/" + year;
        tvDate.setText(date);
    },
    calendar.get(Calendar.YEAR),
    calendar.get(Calendar.MONTH),
    calendar.get(Calendar.DAY_OF_MONTH)
);

dateDialog.show();
```

TimePickerDialog

```
Calendar calendar = Calendar.getInstance();

TimePickerDialog timeDialog = new TimePickerDialog(
    this,
    (view, hourOfDay, minute) -> {
        String time = String.format(
            Locale.getDefault(),
            "%02d:%02d",
            hourOfDay, minute
        );
        tvTime.setText(time);
    },
    calendar.get(Calendar.HOUR_OF_DAY),
    calendar.get(Calendar.MINUTE),
    true
);

timeDialog.show();
```

Current date/time with java.time

```
LocalDate today = LocalDate.now();
LocalTime now = LocalTime.now();

String date = today.toString(); // 2026-10-05
String time = now.toString();
```

Formatting date/time

```
DateTimeFormatter dateFormatter =
        DateTimeFormatter.ofPattern("dd-MM-yyyy");

DateTimeFormatter timeFormatter =
        DateTimeFormatter.ofPattern("hh:mm a");

String date = LocalDate.now().format(dateFormatter);
String time = LocalTime.now().format(timeFormatter);
```

If you use java.time classes, import java.time.LocalDate, java.time.LocalTime and java.time.format.DateTimeFormatter. For very old API levels, use the compatibility approach supported by

your project.

8. SharedPreferences

SharedPreferences stores small key-value data locally, such as login state, username, first-run flags and simple settings. It is not a replacement for a database and should not be used for sensitive secrets such as plaintext passwords.

Create SharedPreferences

```
SharedPreferences prefs =
        getSharedPreferences("MyPrefs", MODE_PRIVATE);
```

Save values

```
SharedPreferences prefs =
        getSharedPreferences("MyPrefs", MODE_PRIVATE);

prefs.edit()
    .putString("username", "Deepak")
    .putBoolean("loggedIn", true)
    .putInt("age", 25)
    .apply();
```

Read values with defaults

```
String username = prefs.getString("username", "");
boolean loggedIn = prefs.getBoolean("loggedIn", false);
int age = prefs.getInt("age", 0);
```

Login-state example

```
SharedPreferences prefs =
        getSharedPreferences("MyPrefs", MODE_PRIVATE);

boolean loggedIn = prefs.getBoolean("loggedIn", false);

if (loggedIn) {
    // Open home screen
} else {
    // Show login screen
}
```

Clear / remove data

```
prefs.edit()
    .remove("username")
    .apply();

// Remove everything in this preference file:
prefs.edit().clear().apply();
```

apply() vs commit()

apply() updates the in-memory value immediately and writes asynchronously. commit() writes synchronously and returns a boolean result. Prefer apply() for normal UI settings unless you specifically need the immediate success result.

9. Explicit and Implicit Intents

An Intent is a message used to request an action from another Android component. Explicit intents identify the target component; implicit intents describe an action and let Android find a suitable app/component.

Explicit Intent — Activity to Activity

```
Intent intent = new Intent(
        MainActivity.this,
        MainActivity2.class
);

intent.putExtra("username", "Deepak");

startActivity(intent);
```

Receive extras in MainActivity2

```
String username = getIntent()
        .getStringExtra("username");

TextView tv = findViewById(R.id.tvUsername);
tv.setText(username);
```

Explicit Intent — return result (modern Activity Result API)

```
ActivityResultLauncher<Intent> launcher =
    registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        result -> {
            if (result.getResultCode() == RESULT_OK
                    && result.getData() != null) {

                String value = result.getData()
                        .getStringExtra("result");
            }
        }
    );

// Launch:
launcher.launch(new Intent(this, MainActivity2.class));
```

Return a result from MainActivity2

```
Intent result = new Intent();
result.putExtra("result", "Success");

setResult(RESULT_OK, result);
finish();
```

Implicit Intent — open browser

```
Intent intent = new Intent(
        Intent.ACTION_VIEW,
        Uri.parse("https://www.google.com")
);

startActivity(intent);
```

Implicit Intent — dial a number

```
Intent intent = new Intent(
        Intent.ACTION_DIAL,
        Uri.parse("tel:9876543210")
);

startActivity(intent);
```

Implicit Intent — send email

```
Intent intent = new Intent(Intent.ACTION_SENDTO);
intent.setData(Uri.parse("mailto:"));
intent.putExtra(Intent.EXTRA_EMAIL,
        new String[]{"student@example.com"});
intent.putExtra(Intent.EXTRA_SUBJECT, "Project");

startActivity(intent);
```

Implicit Intent — share text

```
Intent sendIntent = new Intent(Intent.ACTION_SEND);
sendIntent.setType("text/plain");
sendIntent.putExtra(Intent.EXTRA_TEXT, "Hello Android!");

Intent chooser = Intent.createChooser(
        sendIntent, "Share using"
);

startActivity(chooser);
```

Intent safety

Implicit intents may have no matching application. In production code, consider checking whether an activity can handle the intent before calling startActivity(), especially for optional actions.

Check before launching

```
if (intent.resolveActivity(getPackageManager()) != null) {
    startActivity(intent);
} else {
    Toast.makeText(this,
        "No suitable app found",
        Toast.LENGTH_SHORT).show();
}
```

10. Integrated Mini Lab Example

The following compact pattern combines XML widgets, validation, Spinner, SharedPreferences, Toast and an explicit Intent. It is useful as a lab starting point.

activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Name" />

    <EditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email"
        android:inputType="textEmailAddress" />

    <Spinner
        android:id="@+id/spinnerCourse"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/btnSave"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Save" />
</LinearLayout>
```

MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    EditText etName, etEmail;
    Spinner spinnerCourse;
    SharedPreferences prefs;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etName = findViewById(R.id.etName);
        etEmail = findViewById(R.id.etEmail);
        spinnerCourse = findViewById(R.id.spinnerCourse);
        Button btnSave = findViewById(R.id.btnSave);

        String[] courses = {"MCA", "MSc CS", "BCA"};

        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this,
            android.R.layout.simple_spinner_item,
            courses
        );

        adapter.setDropDownViewResource(
            android.R.layout.simple_spinner_dropdown_item
        );

        spinnerCourse.setAdapter(adapter);

        prefs = getSharedPreferences("StudentPrefs", MODE_PRIVATE);

        btnSave.setOnClickListener(v -> {
            String name = etName.getText().toString().trim();
            String email = etEmail.getText().toString().trim();

            if (name.isEmpty()) {
                etName.setError("Required");
                etName.requestFocus();
                return;
            }

            if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
                etEmail.setError("Invalid email");
                etEmail.requestFocus();
                return;
            }

            String course = spinnerCourse
                    .getSelectedItem().toString();

            prefs.edit()
                .putString("name", name)
                .putString("email", email)
                .putString("course", course)
```

```
                .apply();

            Toast.makeText(
                this,
                "Saved successfully",
                Toast.LENGTH_SHORT
            ).show();
        });
    }
}
```

11. Quick Revision Sheet

Task	Core syntax / method
Find a view	findViewById(R.id.viewId)
Read EditText	editText.getText().toString()
Set text	textView.setText("Hello")
Button click	button.setOnClickListener(v -> { ... });
Checkbox	checkBox.isChecked()
Radio selection	radioGroup.getCheckedRadioButtonId()
Toast	Toast.makeText(this, "msg", Toast.LENGTH_SHORT).show();
Dialog	new AlertDialog.Builder(this).setTitle(...).setMessage(...).show();
ListView	listView.setAdapter(adapter)
Spinner	spinner.setAdapter(adapter)
Fragment replace	getSupportFragmentManager().beginTransaction().replace(...).commit();
Save preference	prefs.edit().putString("key", value).apply();
Read preference	prefs.getString("key", defaultValue)
Explicit intent	startActivity(new Intent(this, TargetActivity.class));
Browser intent	Intent.ACTION_VIEW + Uri.parse(url)
Date picker	new DatePickerDialog(...)
Time picker	new TimePickerDialog(...)

Common imports

```
import android.app.AlertDialog;
import android.app.DatePickerDialog;
import android.app.TimePickerDialog;
import android.content.Intent;
import android.content.SharedPreferences;
import android.graphics.Color;
import android.graphics.drawable.ColorDrawable;
import android.net.Uri;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;

import java.util.Calendar;
import java.util.Locale;
import java.util.regex.Pattern;
import android.util.Patterns;
```

Lab checklist

- Create XML layout and assign IDs to views.
- Call setContentView() before findViewById().

- Connect widgets in Java and attach listeners.
- Validate input before saving or moving to another screen.
- Use ArrayAdapter for simple ListView/Spinner data.
- Use a custom Adapter when each row needs multiple views.
- Use FragmentContainerView + FragmentManager for fragments.
- Use SharedPreferences for small key-value settings, not structured records.
- Use explicit intents for known Activities and implicit intents for system/app actions.
- Test rotation, empty input, invalid input and back navigation.

End of notes — Android Studio + Java + XML